connections running out of *localhost* only. I leave it to you to check out more advanced uses of MySQL.

Since MySQLdb also conforms to the Python DB API 2.0 standard, there are very few changes needed to get our code to work with MySQL. To start with, since MySQL is a full fledged RDBMS, the call to make a connection object is more detailed in comparison to the file based SQLite3. We need to specify the server name (which would be *localhost* when the client runs on the same machine), the user name and password for the account with which we want to access the database, and last of all, the database itself to which we want to connect.

```
>>> import MySQLdb
>>> conn = MySQLdb.connect(host="localhost", user="uname",
passwd="pwd",db="test")
```

Since the cursor object remains the same, the rest of the previous code continues to work well here too, with MySQLdb.

```
>>> c = conn.cursor()
>>> c.execute('''CREATE table workdata (date text, empname
text, projname text,c
ustname text, hours int)''')
0L
>>> conn.commit()
>>> c.execute("INSERT INTO workdata VALUES('01/01/2015',
'john', 'projectA','cus
tomerA',8)")
1L
>>> conn.commit()
```

Further, to do name binding to the row output of the SELECT SQL with SQLite3 we used *sqlite3.Row* for the *row_factory*. With MySQLdb we need to use a different method to do name binding for the result set. We have a specific cursor type defined to be able to access the results as a dictionary object. When we create the cursor object, we need to pass and select the type as *MySQLdb.cursors.DictCursor.*

```
>>> dc = conn.cursor(MySQLdb.cursors.DictCursor)
>>> dc.execute("SELECT * FROM workdata WHERE projname =
'projectA'")
1L
>>> row=dc.fetchone()
>>> type(row)
<type 'dict'>
>>> row['empname']
'john'
```

Note: 1. The *execute()* call returns the number of records affected; we would see later that *executemany()* too does the same.
2. The type of *row* object returned by *fetchone()* is '*dict'*.

The next difference comes at the *executemany()* function. While the SQLite3 version accepted parameter substitution with *?* as placeholder, the MySQLdb version accepts only *%s* as shown below:

```
>>> workrecords = [('01/01/2015','miller','projectA','custom
erA',8),
... ('01/01/2015','john','projectA','customerA',8),
... ('02/01/2015','john','projectA','customerA',8),
... ('02/01/2015','miller','projectB','customerB',8)]
>>> c.executemany('INSERT into workdata VALUES(%s,%s,%s,%s,%s
)',workrecords)
4L
>>> conn.commit()
```

Another difference is in the way we access the results of the query execution. With SQLite3, we could directly iterate over the results of the execute call and do operations with the data retrieved. Instead, with the *dict* cursor, we will have to iterate over the *fetchall()* call.

```
>>> projectAhours=0
>>> c.execute("SELECT * FROM workdata WHERE projname =
'projectA'")
3L
>>> for row in c.fetchall():
...     projectAhours+=row['hours']
...
>>> print "Time spent on ProjectA :"+repr(projectAhours)+ "
hours"
Time spent on ProjectA :24L hours
>>> print "Time spent on ProjectA :"+str(projectAhours)+ "
hours"
Time spent on ProjectA :24 hours
```

Note: The *repr()* call returns a value with an L suffix to indicate it to be long, while the *str()* call returns only the number.

Finally, the query summary function result is also visible below. The difference here is only with respect to how the *repr()* function works, while *str()* remains a better choice.

```
>>> c.execute("SELECT SUM(hours) as totalhours FROM workdata
where projname='pro
jectA'")
1L
>>> sumrow=c.fetchone()
>>> print "Time spent on ProjectA :" +
repr(sumrow['totalhours']) + " hours"
Time spent on ProjectA :Decimal('24') hours
>>> print "Time spent on ProjectA :" +
str(sumrow['totalhours']) + " hours"
Time spent on ProjectA :24 hours
```